MÉTODOS NUMÉRICOS

Práctica de Programación en Java: Arquitectura Modular, Jacobi y Gauss-Seidel

1. Objetivo

Implementar Jacobi y Gauss-Seidel en Java mediante una arquitectura modular que separe la definición del sistema, la lógica de los métodos y la ejecución.

2. Sistema de ecuaciones

```
5x + y + z = 7
x + 4y + z = 6
x + 2y + 5z = 8
```

Vector inicial: X(0) = [0, 0, 0]T. Tolerancia: ε = 0.0001. Máximo: 100 iteraciones.

3. Arquitectura modular

```
Ecuaciones_lineales
■■■ JACOBI
■   ■■■ Defmatrix.java
■   ■■■ MetodoJacobi.java
■   ■■■ MainJacobi.java
■■■ Gauss_Seidel
    ■■■ GaussSeidelMetodo.java
    ■■■ MainGaussSeidel.java
```

4. Fragmentos clave de Jacobi

```
for (int i = 0; i < n; i++) {     double
suma = 0.0;     for (int j = 0; j < n; j++)
{         if (j != i) suma += A[i][j] *
x[j];     }     xNuevo[i] = (b[i] - suma) /
A[i][i]; }

if (calcularError(x, xNuevo) < tol) {
return xNuevo; }
```

5. Fragmentos clave de Gauss-Seidel

```
for (int i = 0; i < n; i++) {     double
suma = b[i];     for (int j = 0; j < n; j++)
{         if (j != i) suma -= A[i][j] *
x[j];     }     x[i] = suma / A[i][i]; }

if (calcularError(xAnt, x) < tol) {
return x; }
```

6. Resultados

Método	Iteraciones	Solución aproximada
Jacobi	16	[0.99998692, 0.99998474, 0.99998256]
Gauss-Seidel	6	[0.99999381, 1.00000015, 1.00000118]

7. Resultados en consola

```
Solución aproximada (Jacobi): [0.99998692, 0.99998474, 0.99998256]
Solución aproximada (Gauss-Seidel): [0.99999381, 1.00000015, 1.00000118]
```

8. Conclusión comparativa

Con ε = 0.0001, Jacobi alcanzó el criterio de parada en 16 iteraciones y Gauss-Seidel en 6. En esta prueba Gauss-Seidel convergió más rápido porque utiliza inmediatamente los valores nuevos calculados durante la misma iteración, mientras que Jacobi utiliza los valores de la iteración anterior.